CN



CNN : Convolutional Neural Network

CNN are a specialized types of neural network designed primarily for processing grid like data, especially images. It is also known as ConvNets. It is inspired by human visual cortex and how it processes visual information.

Why do we need CNN if we have ANN?

- Too many parameters : A regular fully connected neural network treats each pixel as a seperate input.
  - for a small 32x32 RGB image, it has 32x32x3 = 3072 weights for just first hidden layer.
  for 200x200 → 120,000 weights per neuron.
  - This leads to
    - Huge memory requirements
    - Overfitting
    - Very slow training.

- No spatial Information : In ANN, we flatten each pixel of an image in 1D, due to which we cannot get the idea of how close or far are details in an image located.

- No Translation Invariant : A regular NN trained to recognize a cat in the center of an image might fail if the cat appears in the corner.

How CNN's solve these problem :

1. long Connectivity : Neurons only connect to a small local region of input.
2. Parameter Sharing : Same weights (filters) are used across the entire image
3. Translation invariance : Can detect features regardless of their position in the image
4. Fewer Parameters : Much more efficient than fully-connected networks.

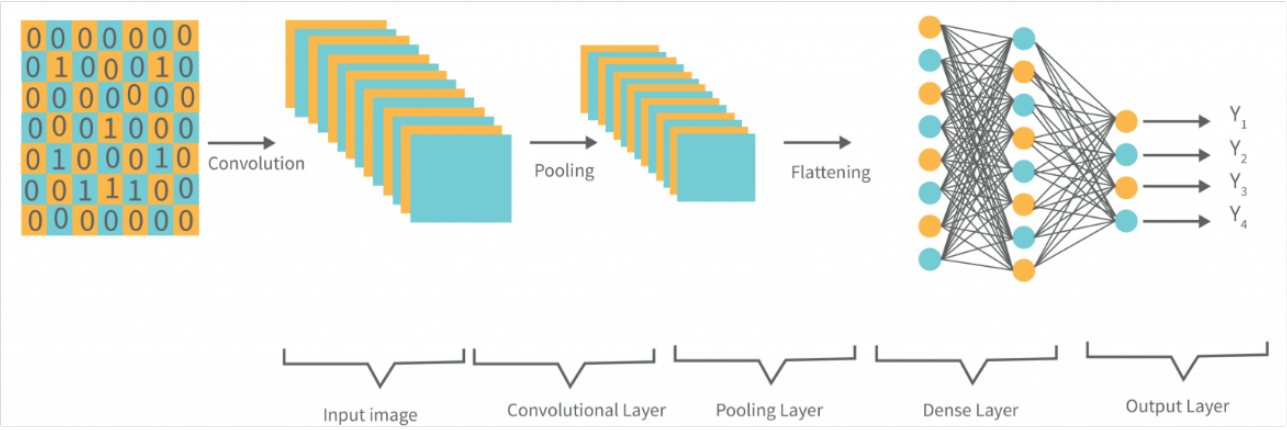
CNN's are used in various applications. Infact, CNN's are one of the most successful model that has been actually used in real life.

- Image Classification
- Object Detection
- Face Recognition
- Medical Image Analysis
- Image Segmentation
- Video analysis
- OCR

Basic Intuition: Takes an image → Extract features automatically → Predict / classify.

- Early layers: Detect simple patterns (edges, colors)
- Middle layers: Combine simple patterns into complex shapes (curves, textures)
- Deep layers: Recognize complete objects (faces, cars, animals)
- Final layers: Make the decision

→ Instead of looking at the pixels at once (too many parameters), CNN looks at regions at a time using filters, building understanding from simple to complex.

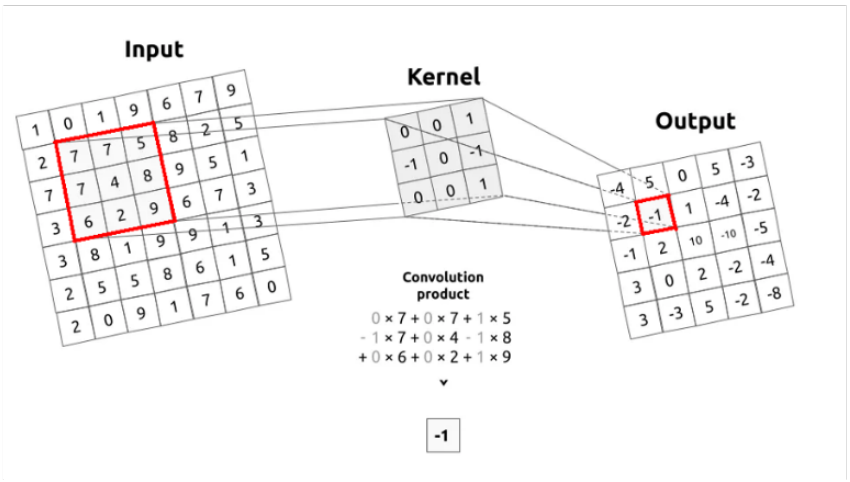


Steps:
1. Input layer
2. Convolution Operation
3. Padding
4. Activation function (ReLU)
5. Pooling layer
6. Repeat multiple times
7. Flatten layer
8. Fully connected layers (Normal ANN)
9. Output layer.

Convolution Operation

Convolution is the fundamental operation in CNN's. It is mathematical operation that applies a filter/kernel to an input image. The filter slides across the image to produce a "feature map" also called activation map

1. Take a filter / kernel : Usually a small matrix (3×3 or 5×5)
2. Place it on input : Start at top-left corner of the input image.
3. Element wise multiplication : Multiply corresponding elements of the filter and the image region.
4. Sum the Results : Add up all the multiplied values to get a single output value.
5. Slide and Repeat : Move the filter to the next position and repeat.



This is simply used to detect edges in an image. Horizontal, vertical or any type of edge you want. It all depends on the filter / kernel.

This works the same for multiple layers. Such as RGB, which has 3 layers, it will need 3 layered filter and will give one layer output.

formula : $n \times n$ (image) $\quad m \times m$ (filter) $\longrightarrow (n-m+1)$

filter / kernel weights are learned during training. (not manually designed)
Each filter is designed to detect a specific feature or pattern.
CNN use multiple features in each convolution layer. Each filter produces one feature map. If we have 32 filters, we get 32 output feature maps.

It detects:

Edge Detection: Horizontal, Vertical, diagonal edges

Color Changes: Transitions from one color to another

Simple patterns: Dots, lines, Corners.

Example Edge detection filters:

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Vertical edge

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

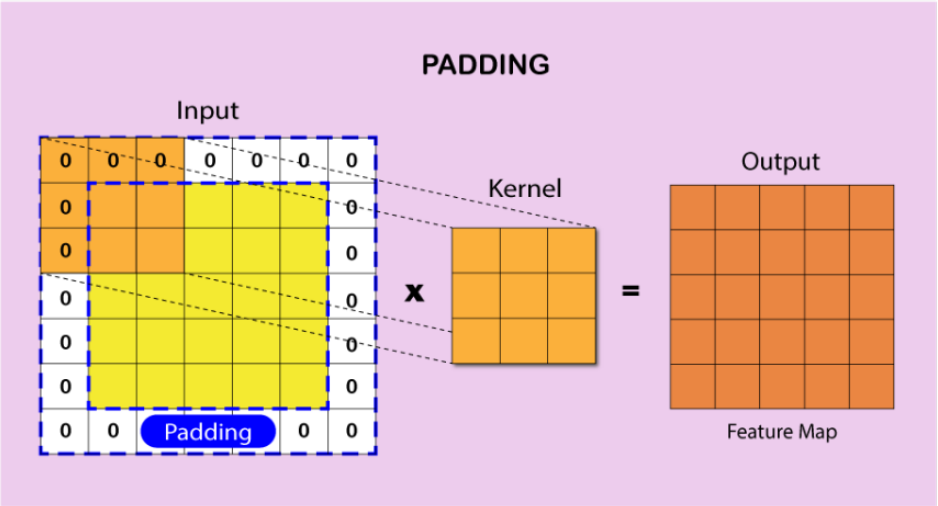
Horizontal edge

$$\begin{bmatrix} -1 & -1 & 0 \\ -1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$$

Diagonal edge

Padding and Strides

Padding is simply "Adding extra border of pixels (usually zeroes) around the input image before applying convolution".



Why do we need padding?

1. Shrinking Output:
   - Every Convolution reduces output dimensions
   - 5x5 * 3x3 filter → 3x3 output (lost 2 pixel on each side
   - After many layers, image becomes too small.
2. Information loss at Border:
   - Corner pixels are used only once in convolution.
   - Center pixels are used multiple times. (they have more importance)
   - Border information gets lost quickly.

formula : $(Input + 2(padding) - filter) + 1$

without padding

```
model = Sequential()
model.add(Conv2D(32,kernel_size=(3,3),padding='valid', activation='relu', input_shape=(28,28,1)))
model.add(Conv2D(32,kernel_size=(3,3),padding='valid', activation='relu'))
model.add(Conv2D(32,kernel_size=(3,3),padding='valid', activation='relu'))

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

```
model.summary()
```

```
Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
conv2d_1 (Conv2D)	(None, 24, 24, 32)	9,248
conv2d_2 (Conv2D)	(None, 22, 22, 32)	9,248
flatten (Flatten)	(None, 15488)	0
dense (Dense)	(None, 128)	1,982,592
dense_1 (Dense)	(None, 10)	1,290

```
Total params: 2,002,698 (7.64 MB)
Trainable params: 2,002,698 (7.64 MB)
Non-trainable params: 0 (0.00 B)
```

with padding

```
model = Sequential()
model.add(Conv2D(32,kernel_size=(3,3),padding='same', activation='relu', input_shape=(28,28,1)))
model.add(Conv2D(32,kernel_size=(3,3),padding='same', activation='relu'))
model.add(Conv2D(32,kernel_size=(3,3),padding='same', activation='relu'))

model.add(Flatten())

model.add(Dense(128, activation='relu'))
model.add(Dense(10, activation='softmax'))
```

```
model.summary()
```

```
Model: "sequential_1"
```

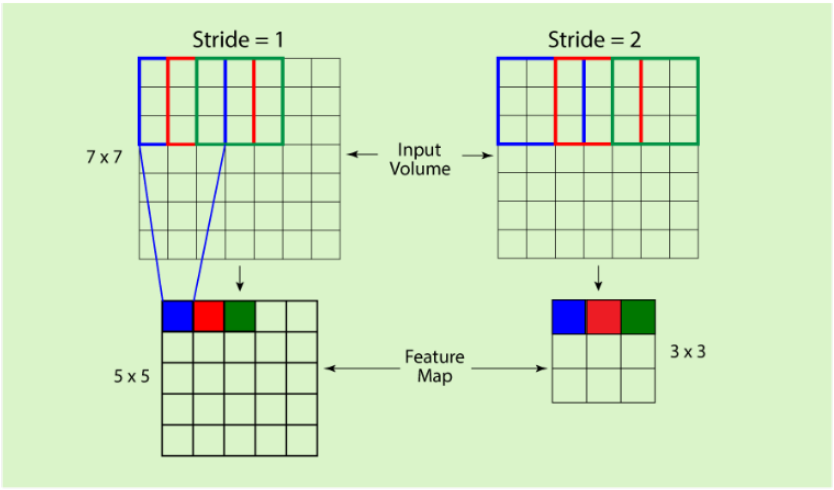
Layer (type)	Output Shape	Param #
conv2d_3 (Conv2D)	(None, 28, 28, 32)	320
conv2d_4 (Conv2D)	(None, 28, 28, 32)	9,248
conv2d_5 (Conv2D)	(None, 28, 28, 32)	9,248
flatten_1 (Flatten)	(None, 25088)	0
dense_2 (Dense)	(None, 128)	3,211,392
dense_3 (Dense)	(None, 10)	1,290

```
Total params: 3,231,498 (12.33 MB)
Trainable params: 3,231,498 (12.33 MB)
Non-trainable params: 0 (0.00 B)
```

Strides

The number of pixels the filter move/jumps at each step during convolution

formula: $\frac{(Input - filter)}{Stride} + 1$



formula of padding + Stride together:

$$Output = \frac{(Input + 2(padding) - Filter)}{Stride} + 1$$

Higher stride simply means → smaller ouptut → fast computation

Why strides are required?

- When you want your model to capture high level features
- If you have less computing power. But nowadays computing power is not an issue anymore. Stride = 1 is nowadays used.

```
model.add(Conv2D(32, kernelsize=(3,3), padding='same', strides = (2,2), activation = 'relu',))
```

Pooling

Pooling is a down sampling operation that reduces the spatial dimensions (height and width) of feature maps while keeping the important information.

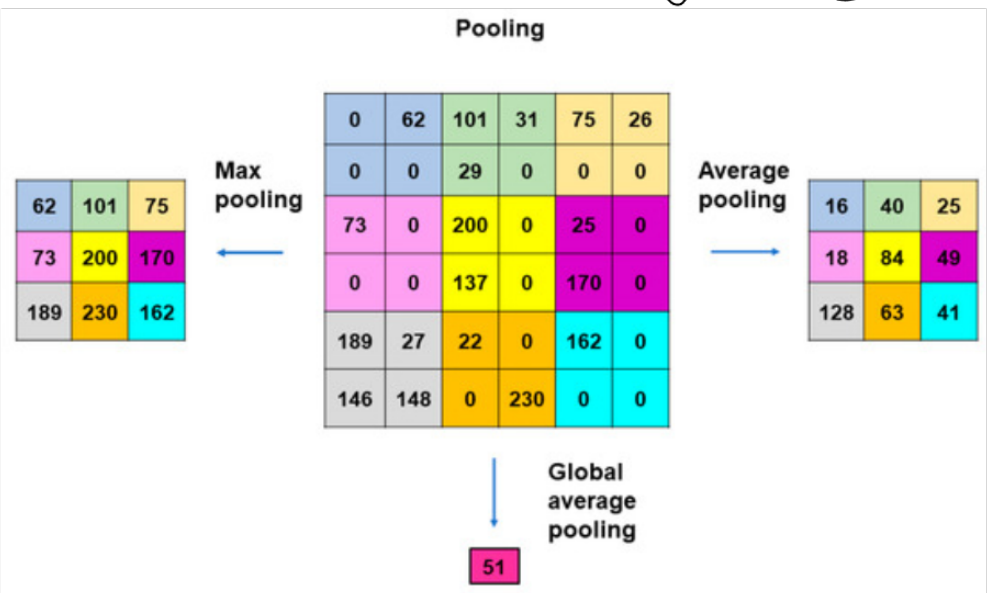
Why do we need pooling?

- The main problem with Convolution is that without pooling, there will be a very large memory which will slow down training.
- Without pooling Network will memorize the data ⟶ Overfitting
- Small translations (details) matter too much.

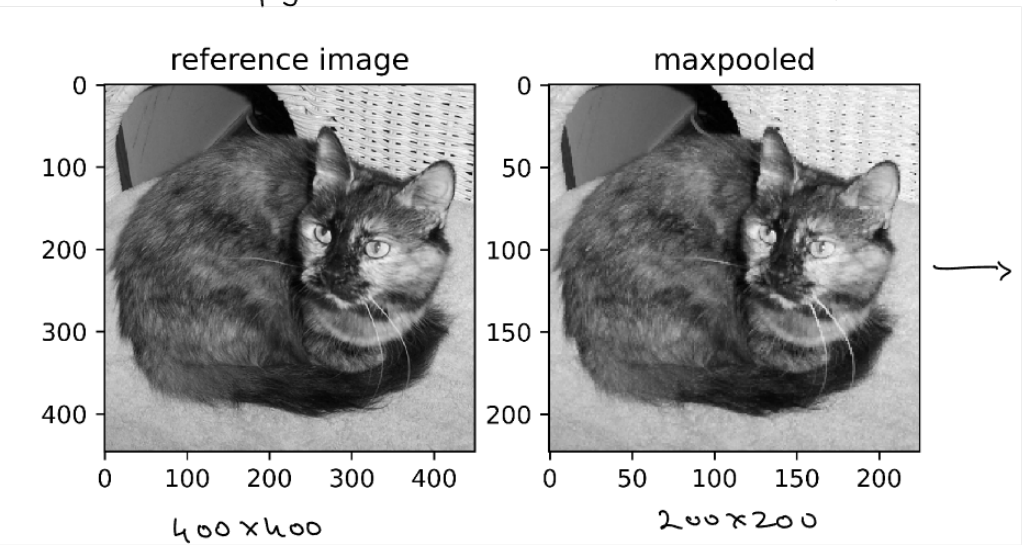
- pooling Reduces Dimensions —> Reducing Parameters
- Translation Invariance : small shift in input dont affect output too much.
  - If we want to detect a cat in an image, by pooling, the location of the cat does not matter as pooling gets the overall overview
- Extracts Dominating Features
- Controls Overfitting. : Acts as regularization

Types of pooling

1. Max pooling
2. Min pooling
3. global pooling
4. Average pooling
5. global average pooling
6. Sum pooling (Rare)



It simply downsamples the image



→ Reduced image quality (But makes sure to get the overall high level view)

Key properties of pooling

- No learnable parameters : It is simply a fixed operation with calculations
- Works on each channel independently
  - pooling is applied seperately to each depth slide.
  - If input has 64 channels, pooling operates on each of 64 feature maps independently.
- Translation Invariance :- Small shift in input dont significantly change output
  - Makes network more robust.
- Information Loss : Pooling is irreversible (you loose data)
  - Can't reconstruct original from pooled output
  - This is good : forces network to learn robust features

Note here that pooling always operates on each channel independently

pooling preserves the depth, only height and width are downscaled.

pooling formula (Universal Output formula)

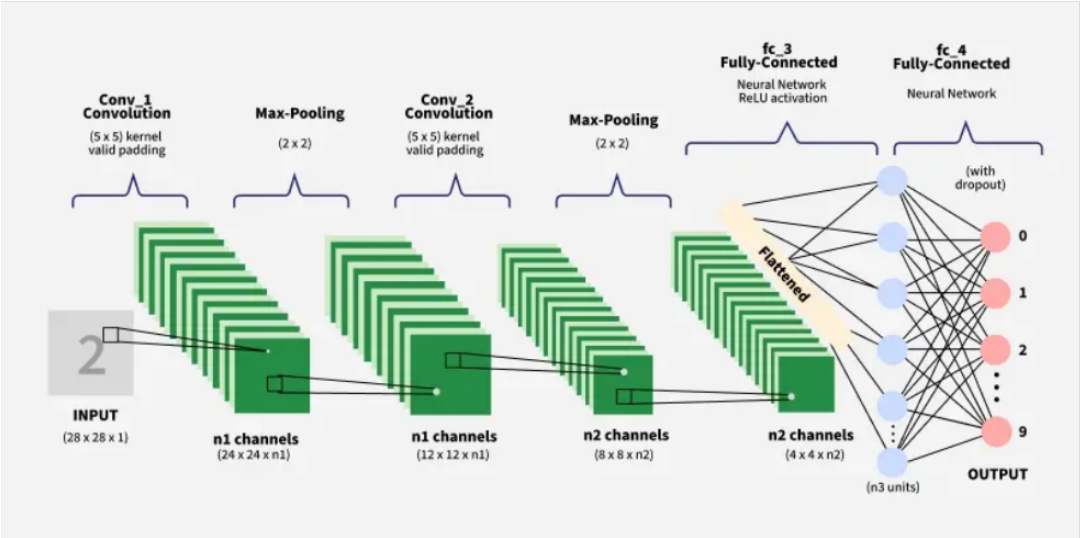
$$\text{Output_height} = \lfloor (\text{Input_Height} + 2(\text{Padding} - \text{Filter_Size}) / \text{Stride} \rfloor + 1$$

$$\text{Output_width} = \lfloor (\text{Input_Width} + 2(\text{Padding} - \text{Filter_Size}) / \text{Stride} \rfloor + 1$$

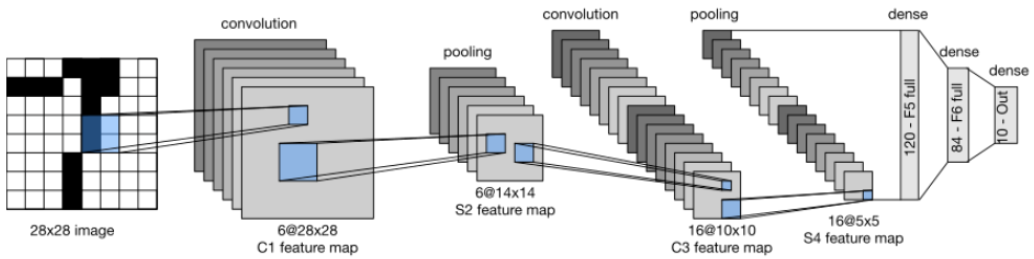
$$\text{Output_Depth} = \text{No. of Filters (for conv) or Input_Depth (for pooling)}$$

$\lfloor \rfloor$ = floor function (values)

CNN Architecture



LeNET - first proposed CNN architecture by Yan Lecun in 1998



CNN vs ANN

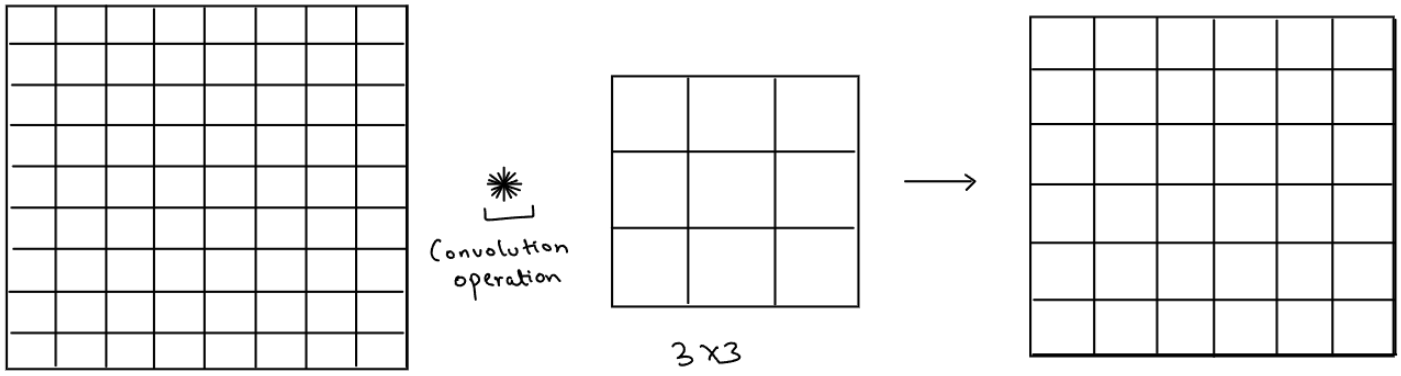
Similarities
- both use back propagation, gradient descent, loss function.
- Same building blocks
- Same goal - minimize loss function and learn from patterns.
- Both have input layers, hidden layers and output layers.

The weights are the filter's value itself in CNN.

The Biggest advantage of CNN
- In ANN, the no. of weights is determined by input neuron x output neuron. So, simply it has direct dependency on input size. As the input grows, the number of learnable parameters grow proportionally.

But in CNN, the input does not matter. It's the filter that decides the total number of learnable parameters. So, even though the input size is large, the computation will be very less because parameters are less.

- While ANN create a unique weight for each input output layer, (making parameters proportional to input size, convolutional layers use shared filters that slide across the input (making parameters independent of input size).

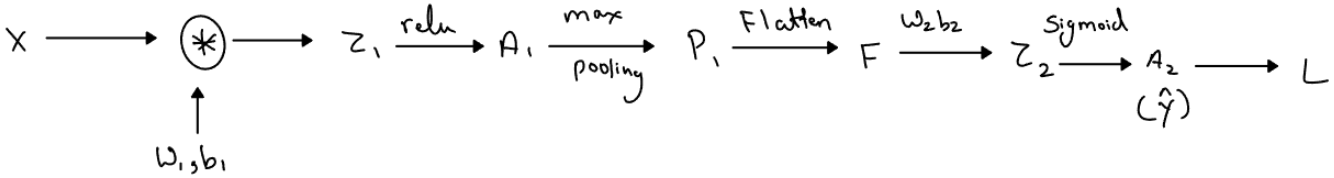


lets say this is 8x8 input

This never changes.

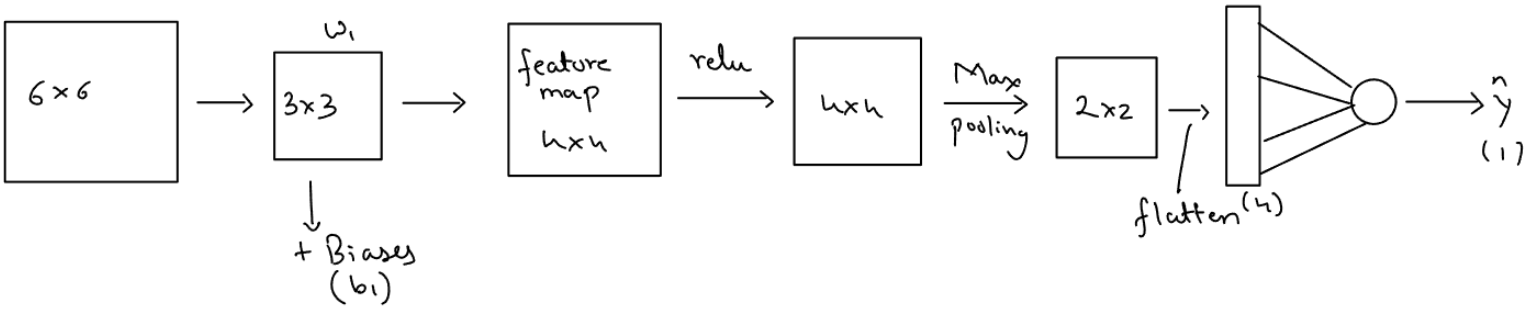
Even if we have 100x100 input, filter has learnable parameters (weights) which are the main focus (parameters)

Simple intuition of CNN.



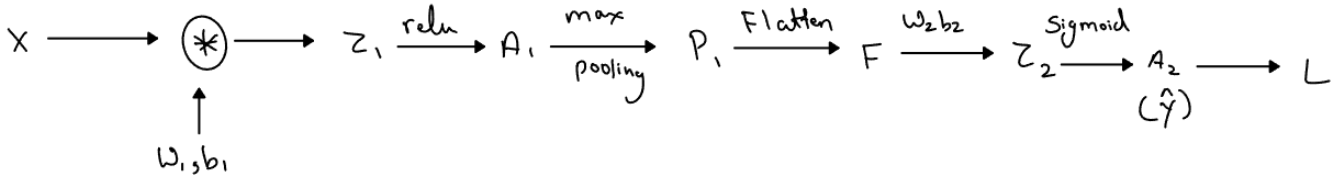
1. Input image is passed
2. Convolution Operation (Using Filters which are learnable parameters)
3. Applying Relu → max(0, x)
4. Applying Max pooling
5. Flattening the Matrix into 1D
6. Weights and biases again
7. Sigmoid Activation function
8. Loss using Gradient Descent Optimizer

Back Propagation



Total Trainable parameters : $W_1 = (3,3)$ $b_1 = (1,1)$ $W_2 = (1,4)$ $b_2 = (1,1)$ = 15 total trainable parameters

Logical Diagram



Here we are taking loss function as Binary cross entropy, since our output is binary here.

Forward propagation

$$Z_1 = Conv(X, W_1) + b_1$$

$$A_1 = relu(Z_1)$$

$$P_1 = \text{max pool}(A_1)$$

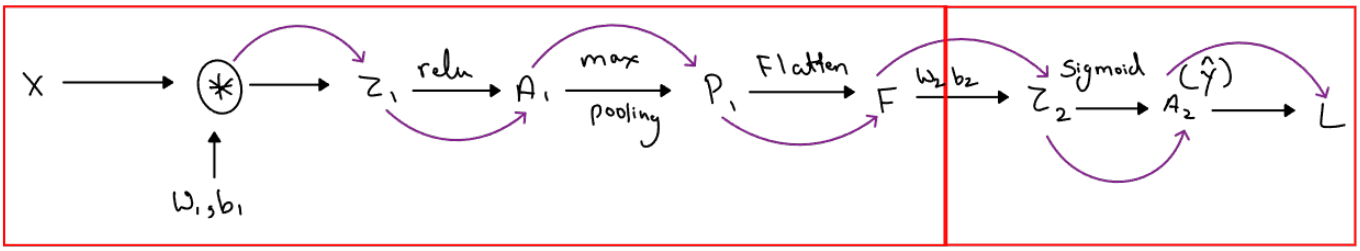
$$F = Flatten(P_1)$$

$$Z_2 = W_2 F + b_2$$

$$A_2 = \sigma(Z_2)$$

Back propagation

Our main goal here is to use gradient descent and minimize the value of Loss function.



for understanding backpropagation better, we have divided the architect into two main parts. One is the Convolution part and other is the neural network part. Firstly we will do neural network part.

In this CNN architect, we have 4 learnable parameter + 1 bias = 5 total

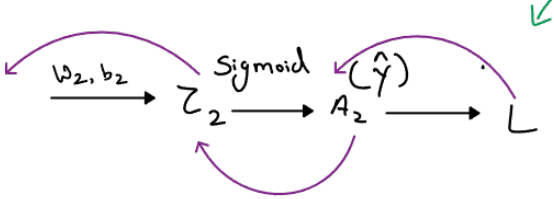
$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

$$b_1 = w_1 - \eta \frac{\partial L}{\partial b_1}$$

$$w_2 = w_2 - \eta \frac{\partial L}{\partial w_2}$$

$$b_2 = b_2 - \eta \frac{\partial L}{\partial b_2}$$

backpropagation part for Neural network



Here, there are

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial Z_2} \times \frac{\partial Z_2}{\partial w_2}$$

$$\frac{\partial L}{\partial b_2} = \frac{\partial L}{\partial A_2} \times \frac{\partial A_2}{\partial Z_2} \times \frac{\partial Z_2}{\partial b_2}$$

Forward propagation formulas: $Z_2 = w_2 F + b_2$

$A_2 = \sigma(Z_2)$

① $$\frac{\partial L}{\partial a_2} = \frac{\partial}{\partial a_2}\left[-y_i \log(a_2) - (1-y_i)\log(1-a_2)\right]$$

$$= -\frac{y_i}{a_2} + \frac{(1-y_i)}{(1-a_2)}$$

$$= -\frac{y_i(1-a_2) + a_2(1-y_i)}{a_2(1-a_2)} = \frac{y_i + y_i a_2 + a_2 - y_i a_2}{a_2(1-a_2)}$$

$$= \frac{a_2 - y_i}{a_2(1-a_2)}$$

② $$\frac{\partial A_2}{\partial Z_2} = \sigma(Z_2)\left[1 - \sigma(Z_2)\right]$$

$$= a_2\left[1 - a_2\right]$$

③ $$\frac{\partial Z_2}{\partial w_2} = F$$ (The initial input layer's inputs)

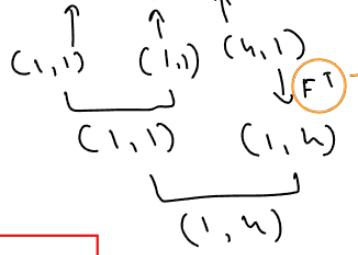
④ $$\frac{\partial Z_2}{\partial b_2} = 1$$

Now finally combining all the equations

$$\frac{\partial L}{\partial w_2} = \frac{(a_2 - y_i)}{a_2(1-a_2)} \times a_2(1-a_2) \times F = (a_2 - y_i) F \approx \boxed{(A_2 - Y) F}$$

$$\frac{\partial L}{\partial b_2} = \frac{(a_2 - y_i)}{a_2(1-a_2)} \times a_2(1-a_2) \times (1) = (a_2 - y_i) \approx (A_2 - Y)$$

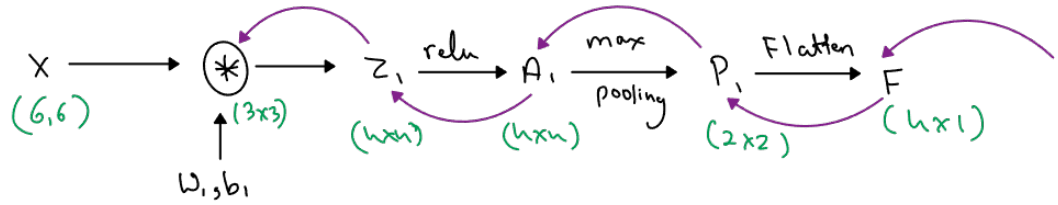
If we focus on the derivative of $\frac{\partial L}{\partial w_2} = (A_2 - Y) F$



We transpose to get the flattened layer.

$$\therefore \boxed{\frac{\partial L}{\partial w_2} = (A_2 - Y) F^T} \qquad \boxed{\frac{\partial L}{\partial b_2} = (A_2 - Y)}$$

Now, for the convolution part



We had 5 total learnable parameters. We have one weight and bias.

Now finding others from the convolution part

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1} \qquad b_1 = w_1 - \eta \frac{\partial L}{\partial b_1}$$

$$\frac{\partial L}{\partial w_1} = \boxed{\frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2}} \frac{\partial Z_2}{\partial F} \frac{\partial F}{\partial P_1} \frac{\partial P_1}{\partial A_1} \frac{\partial A_1}{\partial Z_1} \frac{\partial Z_1}{\partial w_1}$$

$$\frac{\partial L}{\partial b_1} = \boxed{\frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2}} \frac{\partial Z_2}{\partial F} \frac{\partial F}{\partial P_1} \frac{\partial P_1}{\partial A_1} \frac{\partial A_1}{\partial Z_1} \frac{\partial Z_1}{\partial b_1}$$

We know these values from neural part

We have these formulas to calculate from forward propagation :

1] $Z_1 = Conv(X, w_1) + b_1$

2] $A_1 = relu(Z_1)$

3] $P_1 = Maxpool(A_1)$

4] $F = Flatten(P_1)$

5] $Z_2 = w_2 F + b_2$

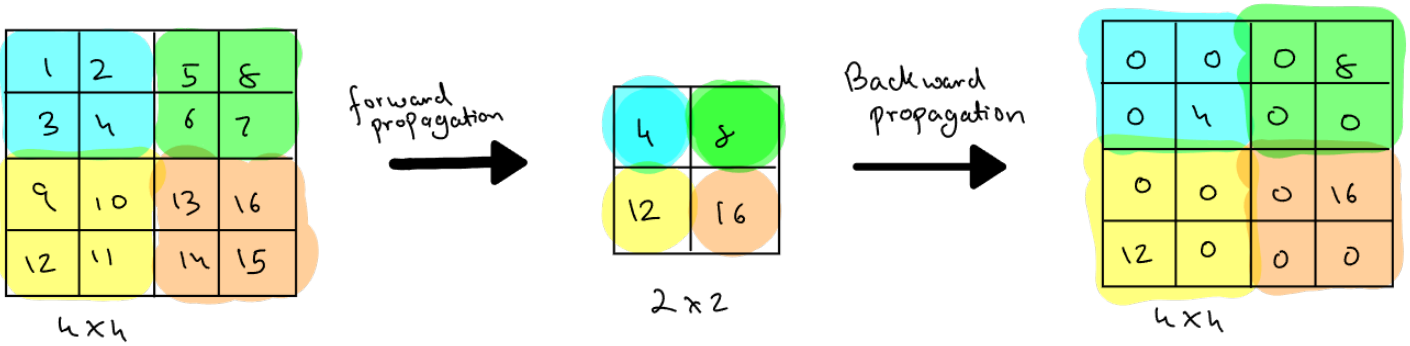
6] $A_2 = \sigma(Z_2)$

④ $\frac{\partial Z_2}{\partial F} = W_2$ (This is direct calculation)

② $\frac{\partial F}{\partial P_1}$ → Now, if you notice, there are no trainable parameters here. And since we are backpropagating, we go from (1,4) → (2,2) So we simply deflatten it to its original version. This is simply done using a function

- reshape (P_1.shape)

③ $\frac{\partial P_1}{\partial A_1}$ → Now here since its max pooling, we have to go from 2x2 to 4x4, Here we simply do not have learnable parameters. And while forward propagating, all the information has been lost other than the max values in max pooling for each grid's small section.
Therefore, you fill all those places with zero, and all the max values original places are still remained the same.



$$\frac{\partial L}{\partial A_2} \frac{\partial A_2}{\partial Z_2} \frac{\partial Z_2}{\partial F} \frac{\partial F}{\partial P_1} \frac{\partial P_1}{\partial A_1} = \frac{\partial L}{\partial A_1}$$

We can say this as $\frac{\partial L}{\partial A_1} = \begin{cases} \frac{\partial L}{\partial P_{1xy}}, & \text{if } A_{mn} \text{ is max element} \\ 0, & \text{otherwise} \end{cases}$

④ $\frac{\partial A_1}{\partial Z_1}$ Since its relu operation

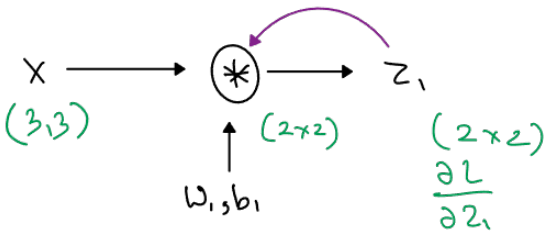
$$= \begin{cases} 1 & \text{if } Z_{1xy} > 0 \\ 0 & \text{if } Z_{1xy} < 0 \end{cases}$$

Now we are at the convolation part. This has learnable parameters. (lets do bias one first)

⑤ $\frac{\partial z_1}{\partial b_1}$

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} \frac{\partial z_1}{\partial b_1}$$

lets find this first



Just changed the values for the sake of simple explaination.

We can write X and w as this (since they are matrix)

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix} \qquad W_1 = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$$

Now when we convolve the filter over X, we get this feature map

$$z_{11} = x_{11}w_{11} + x_{12}w_{12} + x_{21}w_{21} + x_{22}w_{22} + b_1$$

$$z_{12} = x_{12}w_{11} + x_{13}w_{12} + x_{22}w_{21} + x_{23}w_{22} + b_1$$

$$z_{21} = x_{21}w_{11} + x_{22}w_{12} + x_{31}w_{21} + x_{32}w_{22} + b_1$$

$$z_{22} = x_{22}w_1 + x_{23}w_{12} + x_{32}w_{22} + x_{33}w_{22} + b_1$$

$$\frac{\partial L}{\partial z_1} = \begin{bmatrix} \frac{\partial L}{\partial z_{11}} & \frac{\partial L}{\partial z_{12}} \\ \frac{\partial L}{\partial z_{21}} & \frac{\partial L}{\partial z_{22}} \end{bmatrix}$$

from above equations, the value of these derivatives will be 1.

$$\frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z_1} \times \frac{\partial z_1}{\partial b_1}$$

We have this now — find this

Expanding the equation

$$= \frac{\partial L}{\partial z_{11}} \times \frac{\partial z_{11}}{\partial b_1} + \frac{\partial L}{\partial z_{12}} \times \frac{\partial z_{12}}{\partial b_1} + \frac{\partial L}{\partial z_{21}} \times \frac{\partial z_{21}}{\partial b_1} + \frac{\partial L}{\partial z_{22}} \times \frac{\partial z_{22}}{\partial b_1}$$

$$= \frac{\partial L}{\partial z_{11}} + \frac{\partial L}{\partial z_{12}} + \frac{\partial L}{\partial z_{21}} + \frac{\partial L}{\partial z_{22}}$$

$$= \text{Sum}\left(\frac{\partial L}{\partial z_1}\right) \longrightarrow z_1 = \begin{bmatrix} z_{11} & z_{12} \\ z_{21} & z_{22} \end{bmatrix}$$

$$\boxed{\frac{\partial L}{\partial b_1} = \text{Sum}\left(\frac{\partial L}{\partial z_1}\right)}$$

(6) Now lets go for w_1

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$

$$z_{11} = x_{11}w_{11} + x_{12}w_{12} + x_{21}w_{21} + x_{22}w_{22} + b_1$$
$$z_{12} = x_{12}w_{11} + x_{13}w_{12} + x_{22}w_{21} + x_{23}w_{22} + b_1$$
$$z_{21} = x_{21}w_{11} + x_{22}w_{12} + x_{31}w_{21} + x_{32}w_{22} + b_1$$
$$z_{22} = x_{22}w_1 + x_{23}w_{12} + x_{32}w_{22} + x_{33}w_{22} + b_1$$

eq(1)

These are matrix

$$\frac{\partial L}{\partial w_1} = \begin{bmatrix} \frac{\partial L}{\partial w_{11}} & \frac{\partial L}{\partial w_{12}} \\ \frac{\partial L}{\partial w_{21}} & \frac{\partial L}{\partial w_{22}} \end{bmatrix} \qquad \frac{\partial L}{\partial z_1} = \begin{bmatrix} \frac{\partial L}{\partial z_{11}} & \frac{\partial L}{\partial z_{12}} \\ \frac{\partial L}{\partial z_{21}} & \frac{\partial L}{\partial z_{22}} \end{bmatrix}$$

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1}\frac{\partial z_1}{\partial w_1}$$

While doing this differentiation, this terms will also expand because just to find $\frac{\partial L}{\partial w_1}$ we need to find all the derivatives for the inner terms.

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{11}}\times\frac{\partial z_{11}}{\partial w_{11}} + \frac{\partial L}{\partial z_{12}}\times\frac{\partial z_{12}}{\partial w_{11}} + \frac{\partial L}{\partial z_{21}}\times\frac{\partial z_{21}}{\partial w_{11}} + \frac{\partial L}{\partial z_{22}}\times\frac{\partial z_{22}}{\partial w_{11}}$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial z_{11}}\times\frac{\partial z_{11}}{\partial w_{12}} + \frac{\partial L}{\partial z_{12}}\times\frac{\partial z_{12}}{\partial w_{12}} + \frac{\partial L}{\partial z_{21}}\times\frac{\partial z_{21}}{\partial w_{12}} + \frac{\partial L}{\partial z_{22}}\times\frac{\partial z_{22}}{\partial w_{12}}$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_{11}}\times\frac{\partial z_{11}}{\partial w_{21}} + \frac{\partial L}{\partial z_{12}}\times\frac{\partial z_{12}}{\partial w_{21}} + \frac{\partial L}{\partial z_{21}}\times\frac{\partial z_{21}}{\partial w_{21}} + \frac{\partial L}{\partial z_{22}}\times\frac{\partial z_{22}}{\partial w_{21}}$$

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial z_{11}}\times\frac{\partial z_{11}}{\partial w_{22}} + \frac{\partial L}{\partial z_{12}}\times\frac{\partial z_{12}}{\partial w_{22}} + \frac{\partial L}{\partial z_{21}}\times\frac{\partial z_{21}}{\partial w_{22}} + \frac{\partial L}{\partial z_{22}}\times\frac{\partial z_{22}}{\partial w_{22}}$$

We have all the derivatives marked in green.

We have to calculate all the pink ones now using the equation (1)

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{11}}x_{11} + \frac{\partial L}{\partial z_{12}}x_{12} + \frac{\partial L}{\partial z_{21}}x_{21} + \frac{\partial L}{\partial z_{22}}x_{22}$$

derivative

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial z_{11}}x_{12} + \frac{\partial L}{\partial z_{12}}x_{13} + \frac{\partial L}{\partial z_{21}}x_{22} + \frac{\partial L}{\partial z_{22}}x_{23}$$

$$\frac{\partial L}{\partial w_{21}} = \frac{\partial L}{\partial z_{11}}x_{21} + \frac{\partial L}{\partial z_{12}}x_{22} + \frac{\partial L}{\partial z_{21}}x_{31} + \frac{\partial L}{\partial z_{22}}x_{32}$$

$$\frac{\partial L}{\partial w_{22}} = \frac{\partial L}{\partial z_{11}}x_{22} + \frac{\partial L}{\partial z_{12}}x_{23} + \frac{\partial L}{\partial z_{21}}x_{32} + \frac{\partial L}{\partial z_{22}}x_{33}$$

$$X = \begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$$

$$\frac{\partial L}{\partial z_1} = \begin{bmatrix} \frac{\partial L}{\partial z_{11}} & \frac{\partial L}{\partial z_{12}} \\ \frac{\partial L}{\partial z_{21}} & \frac{\partial L}{\partial z_{22}} \end{bmatrix}$$

If we notice,

$$\frac{\partial L}{\partial w_1} = \text{Conv}\left(X, \frac{\partial L}{\partial z_1}\right)$$

Data Augmentation

Data augmentation is simply a technique for artifitially creating new training samples by applying transformations to existing images without actually collecting new data.

Why do we need it?

Deep learning models need a ton of data. If we do not have enough data, data augmentation can help. It also helps reduce overfitting.

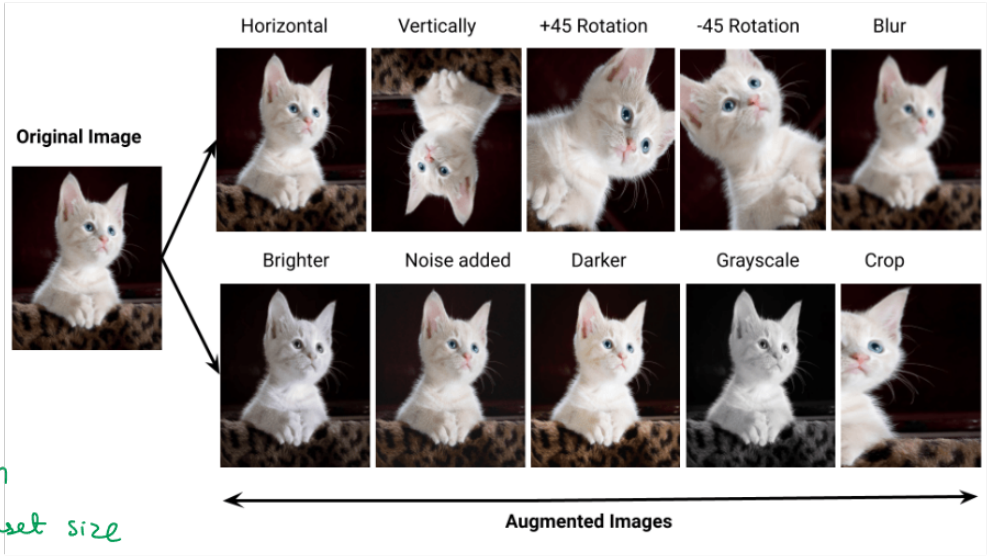
There are various types of transformations

1) Rotation
2) Flipping : Horizontal, Vertical
3) Translation : Shifting
4) Scaling : Zooming
5) Shearing : Slant the image
6) Color Intensity transformations
   - Brightness
   - Contrast
   - Saturation
   - Hue Shift
7) Noise Addition
   - gaussian Noise
   - Salt and pepper noise

Advanced

8) Random cropping
9) Random erasing / cutout
10) Elastic deformations
11) Mixup

And many more.



Benefits

- Prevents Overfitting
- Improves Generalization
- Increases Effective dataset size
- Acts as Regularization
- Cost Effective

Drawbacks

- Increased training time
- Not a replacement of real data
- Risk of unrealistic / Invalid Transformations
- Hyperparameter tuning Complexity is high
- Storage Requirements

Pre trained Models in CNN

In early 2000's, ImageNet was founded where we can find labled images open sourced. (had about 20000 classes)

There was a competition (called ILSVRC) organized for getting the best CV model

Error rate

2010 → ML model → 28 %.

2011 → ML model → 25 %.

2012 → AlexNeT (Geoffrey Hinton) → 16.4 %. [This was the year, a new face of DL was given.]

2013 → ZFNET → 11.7 %.

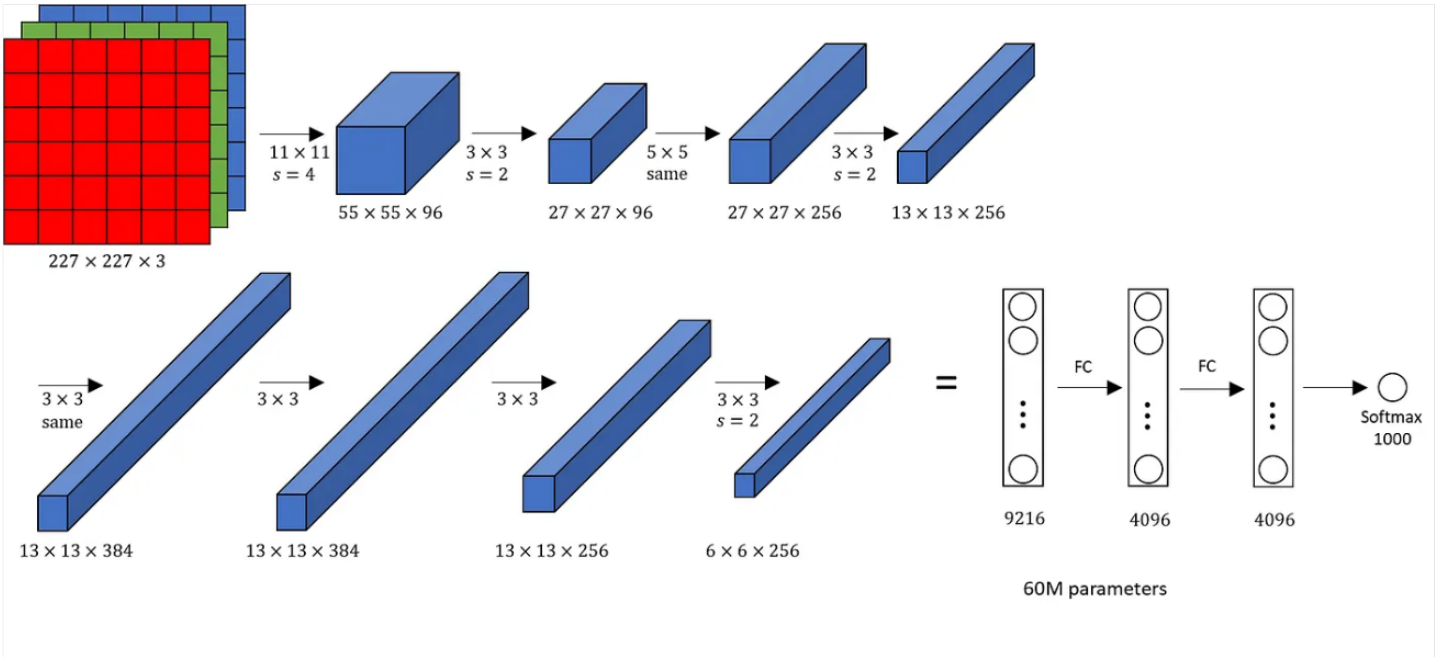
2014 → VGG → 7.3 %.

2015 → GoogleNET → 6.7 %.

2016 → ReSNET → 3.5 %. → surpassing humans.

for more model exploration : https://keras.io/api/applications/

AlexNET Architecture



Transfer Learning

Using a pre-trained model (trained on one task) as a starting point for a different but related task.

Simple analogy: Instead of learning to drive from scratch, you leverage your bicycle skills (balance, traffic rules) to learn driving faster.

So instead of training a CNN from scratch, use a model already trained on millions of images and adapt it to your specific Task.

Problem with training from Scratch: It requires

- millions of labelled images
- Weeks/months of training time
- Expensive GPU's
- Expertise in hyperparameter tuning
- High Computational Cost

Transfer Learning Solutions:

- Works with small datasets
- Trains in hours/days
- Works on consumer GPU's
- Better performance
- Lower computational cost

Hierarchical Feature Learning in CNNs:

```
Early Layers (Low-level features):
- Edges, corners, colors
- Circles, lines, textures
- GENERAL features (useful for ANY image task)

Middle Layers (Mid-level features):
- Simple shapes, patterns
- Object parts (eyes, wheels, leaves)
- SOMEWHAT GENERAL features

Deep Layers (High-level features):
- Complete objects (faces, cars, animals)
- Task-specific features
- SPECIFIC to original task
```

Key Insight:

Early layers learn **universal features** that are useful across different image tasks. Only the final layers are task-specific.

There are few different Techniques for transfer learning

Strategy 1: Feature Extraction : Freeze base model

Strategy 2: Fine tuning : Unfreeze some layers.

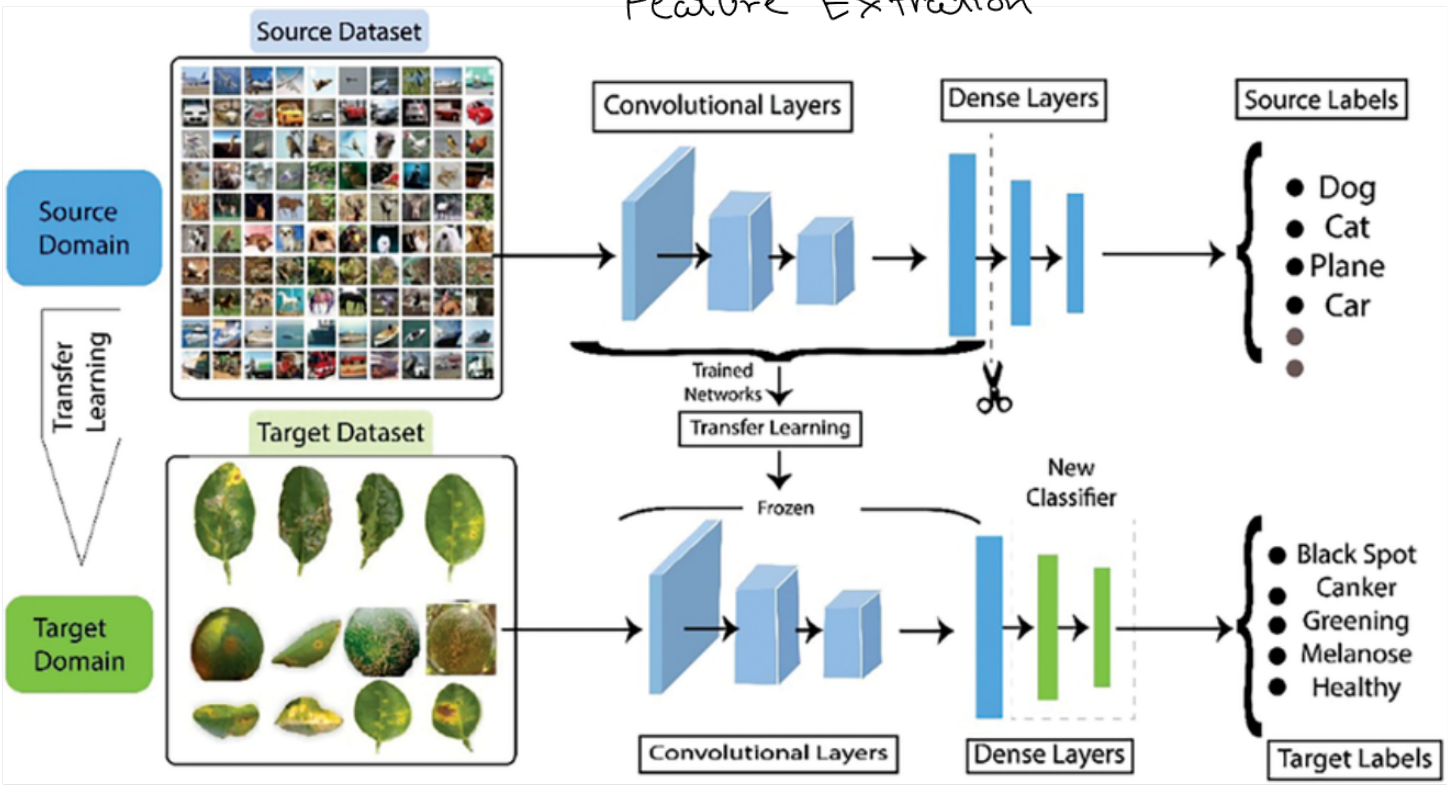
Strategy 3: Full Fine tuning : Unfreeze everything

Decision Framework: Which Strategy to Use?

Based on Dataset Size and Task Similarity:

Your Dataset	Task Similarity	Strategy	Learning Rate
Very Small (100s)	Similar	Feature Extraction	0.001
Very Small (100s)	Different	Feature Extraction + Fine-tune last few	0.0001
Small (1000s)	Similar	Fine-tune top layers	0.0001
Small (1000s)	Different	Feature Extraction or light fine-tuning	0.0001
Medium (10,000s)	Similar	Fine-tune many layers	0.0001
Medium (10,000s)	Different	Fine-tune top layers	0.0001
Large (100,000s+)	Similar	Full fine-tuning	0.00001
Large (100,000s+)	Different	Full fine-tuning or from scratch	0.0001

Feature Extraction



Fine tuning

